



Haute Ecole de la Province de Liège

Techniques avancées de gestion des données

Bases de données avancées et XML

Leçon 2 : DTD et Namespaces

Samuel Hiard

Année académique 2023-2024

Agenda

- DTD
 - Besoin de validation
 - Structure d'un DTD
 - Éléments
 - Attributs
 - Entités
 - Déploiement d'un DTD
- Namespace
 - Besoin
 - Déclaration
 - Portée
 - Utilisation avec un DTD

Agenda

- DTD
 - Besoin de validation
 - Structure d'un DTD
 - Éléments
 - Attributs
 - Entités
 - Déploiement d'un DTD
- Namespace
 - Besoin
 - Déclaration
 - Portée
 - Utilisation avec un DTD

Besoin de validation

- XML est très strict sur certains points
 - contrainte sur les noms, ouverture/fermeture de balise
- et laxiste sur d'autres
 - noms et structure choisis par l'utilisateur
- Ce document XML est-il correct?

```
<?xml version="1.0" encoding="UTF-8" ?>
<coucou>
  <oui pomme="ou non">Va savoir</oui>
  <bim/>
  <etpuis>
    <quoi>Je ne sais pas</quoi>
    <pourquoi>Telle est la question</pourquoi>
  </etpuis>
</coucou>
```

- Toujours possible de vérifier si le document est bien formé (respect des règles inhérentes à XML).
- Besoin d'autre chose pour vérifier la validité ➔ DTD

DTD

- Signifie « Document Type Definition »
- Document spécifiant la structure valide d'un document XML.
- Pour être « valide », un document XML doit respecter son DTD.
- Le DTD est optionnel
 - Dans ce cas, pas de vérification de validité possible
- Il n'est pas le seul document permettant de définir la structure
 - Possibilité d'utiliser un XSD (XML Schema Definition) à la place
 - Nous l'aborderons dans quelques minutes

Agenda

- DTD
 - Besoin de validation
 - Structure d'un DTD
 - Éléments
 - Attributs
 - Entités
 - Déploiement d'un DTD
- Namespace
 - Besoin
 - Déclaration
 - Portée
 - Utilisation avec un DTD

Structure d'un DTD

- La structure d'un DTD est la suivante :
- Déclaration du DOCTYPE
 - Dans le document XML lui-même
 - Précise l'élément root et où trouver le DTD en lui-même
- Déclaration des différents blocs composant la structure
 - Éléments
 - Attributs
 - Entités
 - PCDATA (Parsed Character Data)
 - CDATA (Character Data)

Structure d'un DTD (2)

- Par exemple, un fichier XML contenant un DTD pourrait être celui-ci :

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE note [
  <!ELEMENT note (de, a, sujet)>
  <!ELEMENT de    (#PCDATA)>
  <!ELEMENT a     (#PCDATA)>
  <!ELEMENT sujet (#PCDATA)>
]>
<note>
  <de>moi</de>
  <a>toi</a>
  <sujet>coucou</sujet>
</note>
```

Agenda

- DTD

- Besoin de validation
- Structure d'un DTD
- **Éléments**
- Attributs
- Entités
- Déploiement d'un DTD

- Namespace

- Besoin
- Déclaration
- Portée
- Utilisation avec un DTD

Éléments

- Chaque élément XML doit être déclaré une fois dans le document avec
 - `<!ELEMENT element_name element_content>`
- *element_name* est n'importe quel nom XML valide
- *element_content* est l'une des possibilités suivantes :
 - Un ou plusieurs autres éléments enfants
 - Du texte simple (`#PCDATA`)
 - Un mélange d'éléments et de texte
 - Rien. L'élément est vide.
 - N'importe quoi (`ANY`)

Eléments texte

`<!ELEMENT name (#PCDATA) >`

- On écrit `#PCDATA` entre parenthèses pour spécifier que le contenu est simplement du texte, sans élément enfant.
- `PCDATA` signifie `Parsed Character Data`. Ce terme vient de `SGML`
- Les données sont “parsées” et peuvent contenir des références vers des entités
 - Par exemple, `<` sera remplacé par `<` au moment du traitement
- Il n’est pas possible de contraindre le texte en lui-même à une certaine structure, c’est juste une chaîne de caractères

Eléments enfants

```
<!ELEMENT name (child1, child2, child3)>
```

- Les enfants de l'élément sont indiqués entre parenthèses
- Il s'agit juste d'un exemple, on pourrait imaginer autant d'enfants que l'on souhaite (de 1 $\rightarrow$ ∞)
- L'ordre imposé dans le DTD doit être respecté dans le XML
 - Ceci n'est PAS valide :

```
<name>  
  <child2>Marco</child2>  
  <child1>Paulo</child1>  
  <child3>Ludo</child3>  
</name>
```

- Chaque élément enfant doit, lui-aussi, être défini. Cette définition se fait après la définition de l'élément parent.

Elements enfants : opérateurs

- L' *element_content* peut être une expression régulière précisant la structure
- Dans l'exemple précédent, nous avons utilisé l'opérateur « , » pour indiquer une mise en séquence.
- Il existe 4 autres opérateurs :
 - « | » : choix
 - « ? » : 0 ou 1 occurrence
 - « * » : Répétition d'un nombre quelconque d'occurrences (y compris 0)
 - « + » : Répétition d'un nombre quelconque d'occurrences (au moins 1)

Elements enfants : opérateurs (2)

Quelques exemples :

`<!ELEMENT elem (elem1, elem2, elem3)>`

- L'élément elem doit contenir un élément elem1, un élément elem2 puis un élément elem3 dans cet ordre.

`<!ELEMENT elem (elem1 | elem2 | elem3)>`

- L'élément elem doit contenir un seul des éléments elem1, elem2 ou elem3.

`<!ELEMENT elem (elem1, elem2?, elem3)>`

- L'élément elem doit contenir un élément elem1, un ou zéro élément elem2 puis un élément elem3 dans cet ordre.

`<!ELEMENT elem (elem1, elem2*, elem3)>`

- L'élément elem doit contenir un élément elem1, une suite éventuellement vide d'éléments elem2 et un élément elem3 dans cet ordre.

`<!ELEMENT elem (elem1, (elem2 | elem4), elem3)>`

- L'élément elem doit contenir un élément elem1, un élément elem2 ou un élément elem4 puis un élément elem3 dans cet ordre.

`<!ELEMENT elem (elem1, elem2, elem3)*>`

- L'élément elem doit contenir une suite d'éléments elem1, elem2, elem3, elem1, elem2, ... jusqu'à un élément elem3.

`<!ELEMENT elem (elem1 | elem2 | elem3)*>`

- L'élément elem doit contenir une suite quelconque d'éléments elem1, elem2 ou elem3.

`<!ELEMENT elem (elem1 | elem2 | elem3)+>`

- L'élément elem doit contenir une suite non vide d'éléments elem1, elem2 ou elem3.

Contenu déterministe

- Les expressions qui définissent du contenu doivent être déterministes
 - Il ne faut pas qu'on puisse les interpréter de plusieurs façons.
- En cours d'analyse du contenu, le nom d'un élément ne peut apparaître que sur une seule branche de l'expression.

- Exemple :

```
<!ELEMENT item ((item1, item2) | (item1, item3))>
```

- Que l'on peut remplacer par l'expression déterministe :

```
<!ELEMENT item (item1, (item2 | item3))>
```

- Idem avec « ? » ou « + »
- Par exemple : (item1?, item1) n'est pas déterministe
 - A remplacer par (item1, item1?)
- Il existe toutefois des expressions sans expression déterministe équivalente :

```
<!ELEMENT item ((item1, item2)*, item1)>
```

Eléments mixtes

- Il est possible de combiner élément texte avec élément enfant grâce aux opérateurs.
- `<!ELEMENT name (#PCDATA | child1) *>`
- Il faut toutefois toujours commencer par `#PCDATA`
- On peut alors pour représenter du contenu comportant à la fois du texte et des éléments
- Exemple :

```
<?xml version="1.0" encoding="iso-8859-1"?>
```

```
<!DOCTYPE book [
```

```
<!ELEMENT book (#PCDATA | em | cite)*>
```

```
<!ELEMENT em (#PCDATA)>
```

```
<!ELEMENT cite (#PCDATA)>
```

```
]>
```

```
<book>
```

```
Du <em>text</em>, une <cite>citation</cite> et encore du  
<em>text</em>.
```

```
</book>
```

Élément vide

`<!ELEMENT name EMPTY>`

- L'élément n'a pas de contenu. Il est vide, mais peut avoir des attributs.
- On se focalise seulement sur le contenu de l'élément ici (pas des attributs potentiels).
- Même sans attributs, l'élément en lui-même peut avoir une signification.
 - Comme `<br/>` en XHTML.
- Dans un document XML, on écrit `<name></name>` ou `<name/>`. Il n'y a pas un seul caractère (ni même un espace) entre les balises.

Élément quelconque

`<!ELEMENT name ANY>`

- L'élément peut contenir des éléments enfants et/ou des caractères. Bref, l'élément peut contenir n'importe quoi.
- La seule contrainte, c'est que les éléments aient été déclarés.
- Cela permet d'étendre le document avec du contenu qui n'est pas précisé par le schema.
- Très utile, pendant le développement du DTD, si la structure n'est pas encore clairement définie
- Peut être dangereux, car laisse la porte ouverte à tout et n'importe quoi
 - A n'utiliser que si on sait ce qu'on fait.

Agenda

- DTD

- Besoin de validation
- Structure d'un DTD
- Éléments
- **Attributs**
- Entités
- Déploiement d'un DTD

- Namespace

- Besoin
- Déclaration
- Portée
- Utilisation avec un DTD

Attributs

`<!ATTLIST element_name attribute_definition+>`

- *element_name* être n'importe quel nom legal XML et se réfère à un élément, déclaré ou non.
- *attribute_definition* définit le nom, type et valeur par défaut de l'attribut (voir slide suivant).
- S'il y a plus qu'un ATTLIST pour un élément, ils sont combinés.
- S'il y a plus d'une definition pour un attribut, seul le premier sera pris en compte.

- Exemple :

DTD:

```
<!ELEMENT square EMPTY>
<!ATTLIST square width CDATA "0">
```

XML Valide :

```
<square width="100" />
```

Définition d'attributs

- La partie *attribute_definition* du slide précédent.
- Composé d'un nom d'attribut, d'un type, et d'une valeur
- Nom d'attribut : Un nom XML valide
- Type : Une des valeurs suivantes :
 - CDATA, (en1|en2|..), ID, IDREF, IDREFS, NMTOKEN, NMTOKENS, ENTITY, ENTITIES, NOTATION, xml
- Valeur : Un des suivant :
 - Value, #REQUIRED, #IMPLIED, #FIXED value

Type d'attributs

- CDATA La valeur est une chaîne de caractères
- (en1|en2|..) La valeur est une liste énumérée
- ID La valeur est un id unique
- IDREF La valeur est l'id d'un autre élément
- IDREFS La valeur est une liste d'autres ids
- NMTOKEN La valeur est un nom XML valide
- NMTOKENS La valeur est une liste de noms XML valides
- ENTITY La valeur est une entité
- ENTITIES La valeur est une liste d'entités
- NOTATION La valeur est le nom d'une notation
- xml: La valeur est une valeur xml prédéfinie

Valeur des attributs

- value La valeur par défaut de l'attribut
- #REQUIRED L'attribut est obligatoire
- #IMPLIED L'attribut est optionnel
- #FIXED value La valeur de l'attribut est fixée

Quelques exemples

XML

```
<chapter name="chapter.introduction">
Introduction</chapter>3<p>See chapter <ref
name="chapter.introduction"/> ...</p>
```

DTD

```
<!ATTLIST chapter name ID #REQUIRED>
<!ATTLIST ref name IDREF #REQUIRED>
```

Quelques exemples (2)

XML

```
<entity name="client">...</entity>
<entity name="employee">...</entity>
<generalization name="person" entities="client
employee">
...
</generalization>
```

DTD

```
<!ATTLIST entity name ID #REQUIRED>
<!ATTLIST generalization name ID #REQUIRED entities
IDREFS #REQUIRED>
```

Quelques exemples (3)

XML

```
<test test1=" name " test2=" na me "/>
```

DTD

```
<!ATTLIST test test1 NMTOKEN #REQUIRED test2 CDATA  
#REQUIRED>
```

Quelques exemples (4)

XML

```
<schedule day="thursday"/>
```

DTD

```
<!ATTLIST schedule day ( monday | tuesday |  
wednesday | thursday | friday | saturday | sunday )  
#REQUIRED>
```

Attributs vs Éléments

- Quand utiliser l'un ou l'autre?
- Techniquement, un attribut et un élément de type text peuvent être interchangeables.
- Ceci peut être valide (même si déconseillé) :

```
<note de="moi" a="toi" sujet="coucou"></note>
```

- Il n'existe pas de contraintes réelles, mais des règles de bonne pratique
- De manière générale :
 - Données : Dans les éléments
 - Meta-données : Dans les attributs

Agenda

- DTD

- Besoin de validation
- Structure d'un DTD
- Éléments
- Attributs
- **Entités**
- Déploiement d'un DTD

- Namespace

- Besoin
- Déclaration
- Portée
- Utilisation avec un DTD

Entités

- Possibilité de définir des entités
 - autre que les entités prédéfinies < > ...

- 2 types:

- **Entités textuelles**

```
<!ENTITY nom "contenu">
```

Exemple dans DTD

```
<!ENTITY signature "Cordialement, Samuel Hiard">
```

Et dans XML

```
<note>Bonjour, Merci pour tout. &signature;</note>
```

- **Entité paramètres**

```
<!ENTITY % nom "contenu">
```

Exemple dans DTD

```
<!ENTITY % nom "nom CDATA #REQUIRED">
```

```
<!ENTITY % prenom "prenom CDATA #IMPLIED">
```

```
<!ATTLIST personne %nom; %prenom;>
```

Agenda

- DTD
 - Besoin de validation
 - Structure d'un DTD
 - Éléments
 - Attributs
 - Entités
 - Déploiement d'un DTD
- Namespace
 - Besoin
 - Déclaration
 - Portée
 - Utilisation avec un DTD

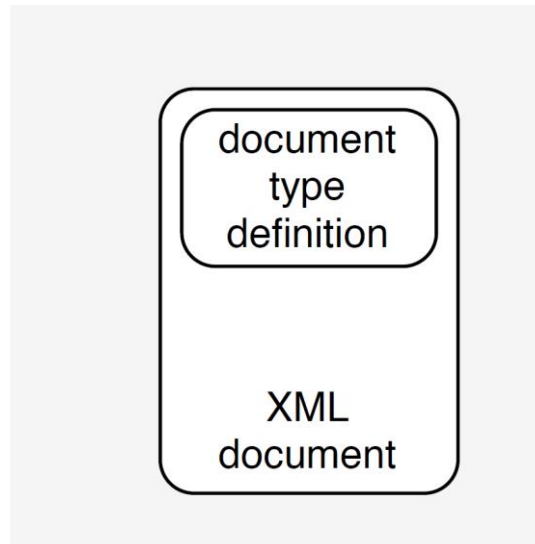
Localisation d'un DTD

- Un DTD peut être
 - Interne (défini dans le document)
 - Externe avec SYSTEM (défini à l'extérieur du document)
 - Externe avec PUBLIC (défini à l'extérieur du document)
 - Mixte (Interne + Externe)
- La syntaxe que nous allons voir ici s'applique aussi aux entités!
 - Qui peuvent donc être interne ou externe (SYSTEM ou PUBLIC)

DTD Interne

```
<!DOCTYPE root_element [ ... ]>
```

- Le DTD est contenu à l'intérieur du document.
- Utile si on veut un document qui se satisfait à lui-même



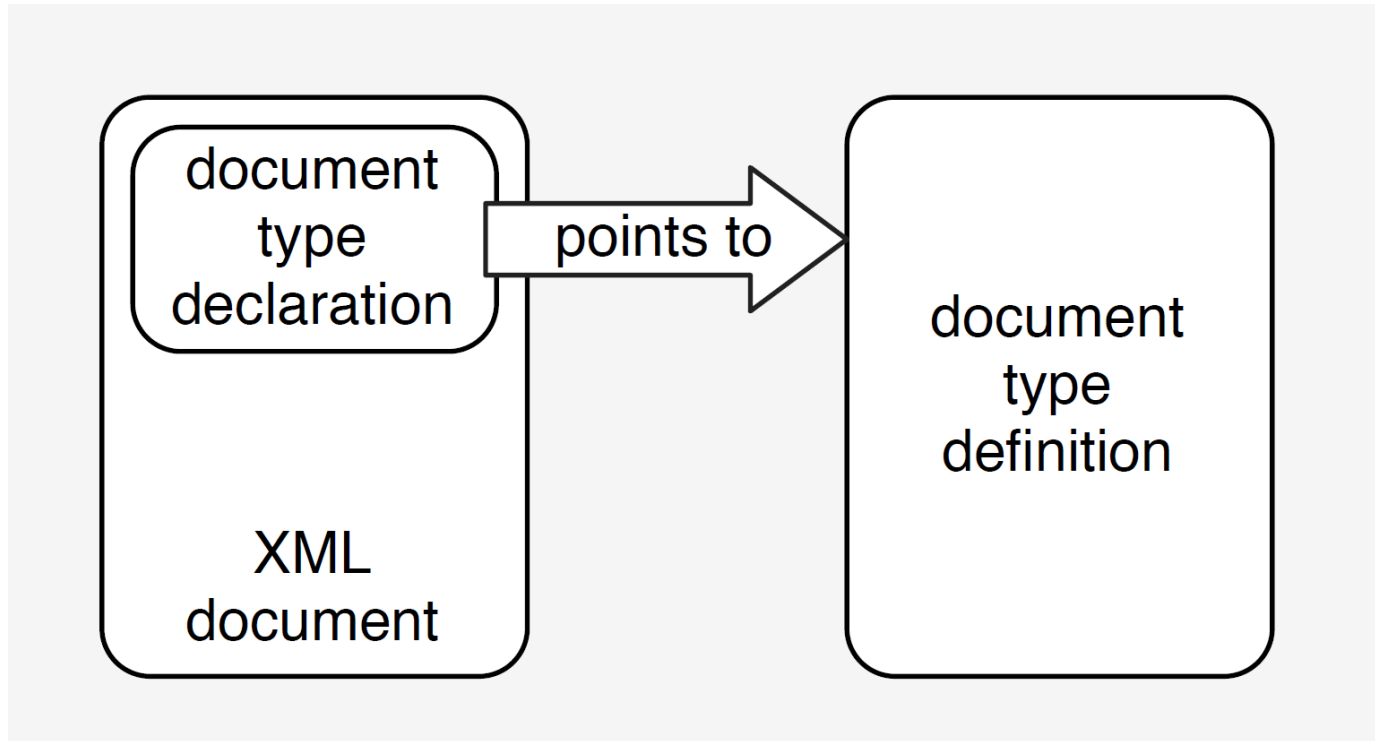
DTD externe avec SYSTEM

```
<!DOCTYPE root_element SYSTEM "URI
```

- Le DTD est défini dans un document à part
- Séparation entre déclaration et définition
- Usage traditionnel du DTD
- L'URI est l'adresse où se trouve la définition du DTD

DTD externe avec SYSTEM

- Graphiquement :



DTD externe avec PUBLIC

```
<!DOCTYPE root_element PUBLIC "Unique_id"  
"Backup_URI">
```

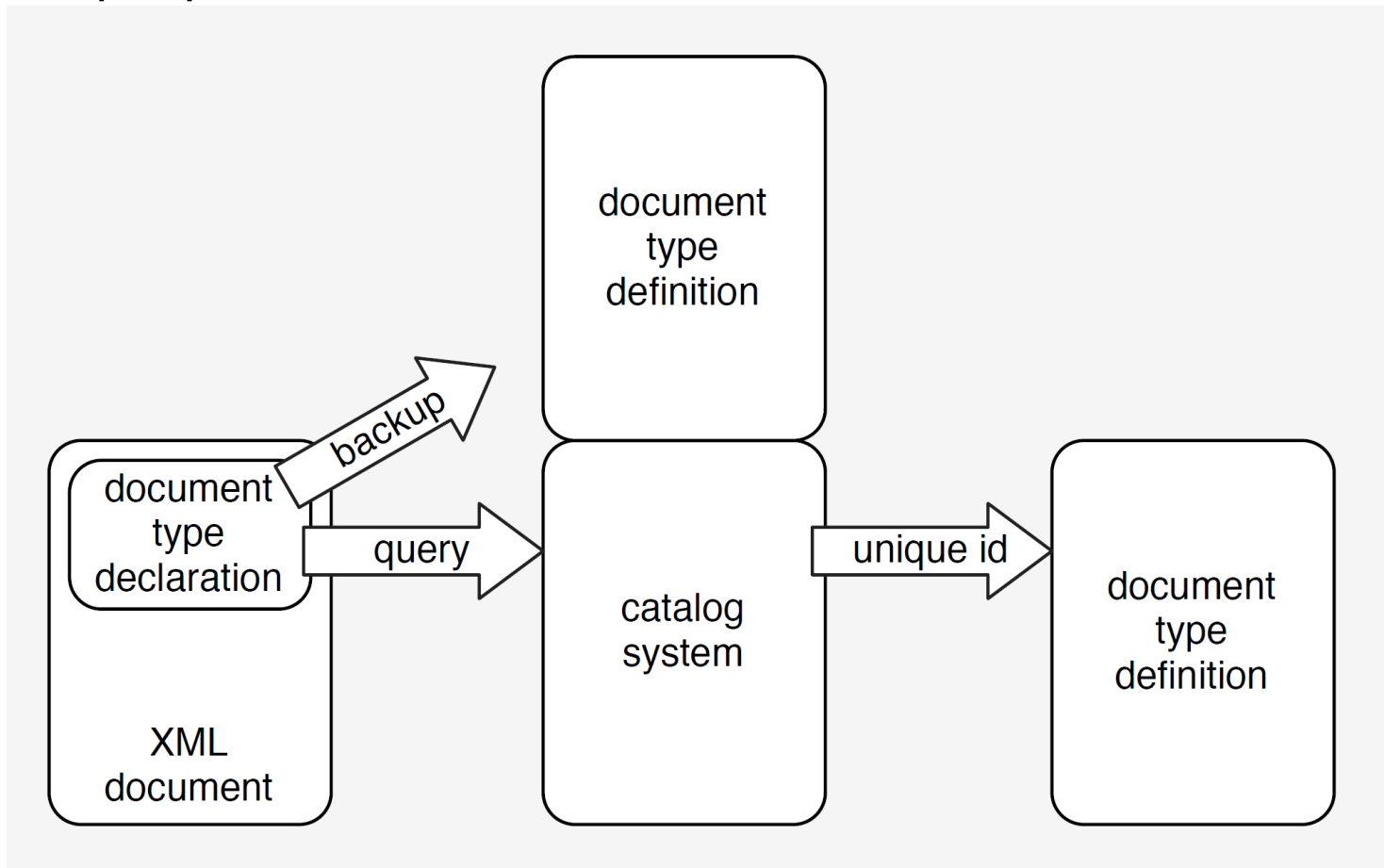
- Le DTD est défini dans un document à part
- Un système de catalogue nous aide à localiser le DTD.
- Comme le DOCTYPE de XHTML:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"  
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

- *Backup_URI* dans le cas où on ne trouverait pas le document à l'aide du catalogue.
- Séparation entre déclaration et définition

DTD externe avec PUBLIC

- Graphiquement :



DTD Mixte

```
<!DOCTYPE root_element SYSTEM "URI" [...] >
```

```
<!DOCTYPE root_element PUBLIC "Unique_ID" SYSTEM  
"Backup_URI" [ ... ] >
```

- Le DTD est principalement à l'extérieur du document XML mais aussi à l'intérieur.
- Possibilité d'étendre le DTD avec des nouveaux éléments, attributs, entités.
- La déclaration la plus récente (interne) prend la priorité
 - Possibilité de redéfinir les éléments du DTD externe

Exemple de DTD Mixte

- DTD original :

```
<!ENTITY % book.entries "">
<!ELEMENT bibliography (book)+>
<!ELEMENT book (title, author, year, publisher, isbn
%book.entries;)>
<!ATTLIST book key NMTOKEN #REQUIRED>
<!ATTLIST book lang (fr | en) #IMPLIED>
<!ELEMENT title (#PCDATA)>
<!ELEMENT author (#PCDATA)>
<!ELEMENT year (#PCDATA)>
<!ELEMENT publisher (#PCDATA)>
<!ELEMENT isbn (#PCDATA)>
```

Exemple de DTD Mixte (2)

- Redéfinition du DTD :

```
<?xml version="1.0" encoding="iso-8859-1"
standalone="no"?>
<!DOCTYPE bibliography SYSTEM "bibliography.dtd" [
<!-- Ajout d'un enfant url à l'élément book -->
<!ENTITY % book.entries ", url?">
<!-- Redéclaration de l'attribut lang de l'élément
book -->
<!-- L'attribut devient obligatoire -->
<!ATTLIST book lang (fr | en) #REQUIRED>
<!-- Déclaration de l'élément url -->
<!ELEMENT url (#PCDATA)>
]>
```

Exemple de DTD Mixte (3)

- Ce contenu XML est alors valide:

```
<bibliography>
<book key="Michard01" lang="fr">
<title>XML langage et applications</title>
<author>Alain Michard</author>
<year>2001</year>
<publisher>Eyrolles</publisher>
<isbn>2-212-09206-7</isbn>
<url>http://www.editions-eyrolles/livres/michard/</url>
</book>
<book key="Marchal00" lang="fr">
<title>XML by Example</title>
<author>Benoît Marchal</author>
<year>2000</year>
<publisher>Macmillan Computer Publishing</publisher>
<isbn>0-7897-2242-9</isbn>
</book>
</bibliography>
```

Agenda

- DTD
 - Besoin de validation
 - Structure d'un DTD
 - Éléments
 - Attributs
 - Entités
 - Déploiement d'un DTD
- Namespace
 - Besoin
 - Déclaration
 - Portée
 - Utilisation avec un DTD

Le problème des noms

Imaginons le document suivant :

```
<societe>
  <nom>Genius S.A.</nom>
  <adresse> 113, Rue de la réussite 4500 Visé</adresse>
  <contact>
    <nom>Bidendum</nom>
    <prenom>Albert</prenom>
    <fonction>PDG</fonction>
  </contact>
</societe>
```

Le DTD contiendra donc la ligne suivante :

```
<!ELEMENT nom (#PCDATA)>
```

Mais de quel nom parle-t-on? Celui de la société ou celui du contact?

Solution

- Si tout se trouve dans notre DTD, il suffit alors de changer l'un des deux noms
 - On aura, par exemple `nomContact`
- Par contre, si on souhaite combiner plusieurs DTD externes, il faudrait alors redéfinir l'un des deux
 - Avec une déclaration mixte, par exemple
 - Pénible
 - Source d'erreurs
- On préférera alors utiliser alors un *namespace* (espace de noms) pour différencier les différents domaines.

Agenda

- DTD
 - Besoin de validation
 - Structure d'un DTD
 - Éléments
 - Attributs
 - Entités
 - Déploiement d'un DTD
- Namespace
 - Besoin
 - Déclaration
 - Portée
 - Utilisation avec un DTD

Déclaration d'un namespace

- Un *namespace* sera déclaré dans un élément avec la attribut `xmlns`

```
<name xmlns:namespace="URI">
```

- *name* est le nom de l'élément
- *namespace* est le nom du namespace
- *URI* est un identifiant en rapport avec le namespace
 - Il est simplement utilisé comme clé unique
 - Il peut pointer vers un site web décrivant le namespace...
 - ... ou vers rien du tout !
- Exemple :

```
<societe xmlns:contact=" http://monsite.be/namespace.html">
```

Fichier XML avec namespace

- On pourra alors avoir ceci :

```
<societe>
  <nom>Genius S.A.</nom>
  <adresse> 113, Rue de la réussite 4500 Visé</adresse>
  <contact xmlns:c="http://contact.be">
    <c:nom>Bidendum</c:nom>
    <c:prenom>Albert</c:prenom>
    <c:fonction>PDG</c:fonction>
  </contact>
</societe>
```

Agenda

- DTD
 - Besoin de validation
 - Structure d'un DTD
 - Éléments
 - Attributs
 - Entités
 - Déploiement d'un DTD
- Namespace
 - Besoin
 - Déclaration
 - Portée
 - Utilisation avec un DTD

Portée d'un namespace

- Un namespace peut être référencé depuis l'élément qui le définit ainsi que depuis ses enfants.
- L'exemple précédent aurait pu s'écrire :

```
<societe>
  <nom>Genius S.A.</nom>
  <adresse> 113, Rue de la réussite 4500 Visé</adresse>
  <c:contact xmlns:c="http://contact.be">
    <c:nom>Bidendum</c:nom>
    <c:prenom>Albert</c:prenom>
    <c:fonction>PDG</c:fonction>
  </c:contact>
  <!-- Le namespace c n'existe plus à partir d'ici -->
</societe>
```

Portée d'un namespace (2)

Il est alors possible de modifier le namespace en cours de route

```
<s:societe xmlns:s="http://societe.be">
  <s:nom>Genius S.A.</s:nom>
  <s:adresse> 113, Rue de la réussite 4500 Visé</s:adresse>
  <c:contact xmlns:c="http://contact.be">
    <c:nom>Bidendum</c:nom>
    <c:prenom>Albert</c:prenom>
    <c:fonction>PDG</c:fonction>
  </c:contact>
</s:societe>
```

Namespace par défaut

- Ce namespace n'a pas réellement de nom
 - Ce que nous utilisons avant d'avoir conscience de l'existence des namespaces
- Il est possible de retourner au namespace par défaut en utilisant l'attribut xmlns sans donner de nom
- Exemple :

```
<s:societe xmlns:s="http://societe.be">
  <s:nom>Genius S.A.</s:nom>
  <s:adresse> 113, Rue de la réussite 4500 Visé</s:adresse>
  <contact xmlns="">
    <nom>Bidendum</nom>
    <prenom>Albert</prenom>
    <fonction>PDG</fonction>
  </contact>
</s:societe>
```

Agenda

- DTD
 - Besoin de validation
 - Structure d'un DTD
 - Éléments
 - Attributs
 - Entités
 - Déploiement d'un DTD
- Namespace
 - Besoin
 - Déclaration
 - Portée
 - Utilisation avec un DTD

DTD et namespace

- Les DTD ne prennent pas compte les espaces de noms. Il est cependant possible de valider, avec une DTD, un document avec des espaces de noms au prix de quelques entorses à l'esprit des espaces de noms.
- Il y a, en effet, quelques contraintes.
- D'une part, la déclaration d'espace de noms est vue comme un attribut de nom commençant par xmlns:. Il est donc nécessaire de le déclarer comme tout autre attribut.
- D'autre part, les noms qualifiés des éléments sont considérés comme des noms contenant le caractère ':'. Il faut donc déclarer les éléments avec leur nom qualifié

DTD et namespace (2)

Exemple valide

- DTD

```
<!-- Fichier "valid.dtd" -->  
<!ELEMENT tns:list (tns:item)+>  
<!ATTLIST tns:list xmlns:tns CDATA #REQUIRED>  
<!ELEMENT tns:item (#PCDATA)>
```

- XML

```
<?xml version="1.0" encoding="iso-8859-1"?>  
<!DOCTYPE list SYSTEM "valid.dtd">  
<tns:list xmlns:tns="http://www.omega-one.org/~carton/">  
    <tns:item>Item 1</tns:item>  
    <tns:item>Item 2</tns:item>  
</tns:list>
```

DTD et namespace (3)

- Par contre, le fichier XML suivant n'est pas valide par rapport à son DTD, même si nous avons simplement changé le préfixe :

```
<?xml version="1.0" encoding="iso-8859-1"?>
<!DOCTYPE list SYSTEM "valid.dtd">
<ons:list xmlns:ons="http://www.omega-one.org/~carton/">
    <ons:item>Item 1</ons:item>
    <ons:item>Item 2</ons:item>
</ons:list>
```

- Il faudrait donc un schema qui prenne en compte la notion de namespace
 - XML Schema Definition (XSD)
 - Nous le verrons la semaine prochaine !